

# Reinforcement learning for flight control

Kušnirik, Ondrej  
`xkusni02@stud.fit.vutbr.cz`

Greš, Daniel  
`xgresd00@stud.fit.vutbr.cz`

Kvaček, David  
`xkvace00@stud.fit.vutbr.cz`

April 2026

The aim of this work is to train an adaptive deep neural network that is going to be able to control various types of fixed-wing planes without the knowledge of the plane's parameters upfront. The simulation will be handled by the JSBSim Flight Dynamics Model which lies at the core of FlightGear, an open-source flight simulator. We would like to first try standard reinforcement learning methods which others have used successfully for this type of problem to create a base for further evaluation of our own work (e.g. DDPG – Deep Deterministic Policy Gradient<sup>1</sup> based agent). As for the environment, we would like to utilize Gymnasium, OpenAI's Python library, which is being used extensively for reinforcement learning tasks. We will be building upon the work of Mr. Galego<sup>2</sup> who has already connected the FlightGear to the Gymnasium environment. Mr. Galego tried to train the PPO RL agent for the task of flying with constant heading and altitude; however, the agent seems to develop a rather strange behavior in which the plane shakes during the flight excessively. Our contribution will be to understand the principles of the reward function and the RL method used and possibly tweak them to achieve better results. The solution is divided into phases, from the launch of the JSBSim simulator within the Gymnasium library, through the definition of the 3D waypoint system, modification of the observation space relative to the next point, and modification of the reward function to motivate the fastest possible flight path, to experiments with more advanced algorithms, verification of the agent's ability to control a flying device, and final measurement of the flight path success, including performance comparison<sup>3</sup>.

---

<sup>1</sup><https://link.springer.com/article/10.1007/s11071-023-08725-y>

<sup>2</sup><https://github.com/JGalego/gymnasium-jsbsim/tree/master>

<sup>3</sup><https://github.com/Humblesaw/knn>

# 1 Introduction

The goal of this project is to teach a neural network to fly a plane through an arbitrary set of points in space. To do this we utilize different reinforcement learning algorithms, thus we need to define the basic environment specifics and a common evaluation function.

## 1.1 Observation space

The observation space consists of 9 variables which are the observed results of the plane's behavior in the simulation environment (JSBSim), 4 variables which the model directly controls and 3 variables which are calculated based on the observed behavior.

### Observed variables

- **position/h-sl-ft** – Altitude above mean sea level [ft].
- **attitude/pitch-rad** – Pitch of the aircraft (nose up/down) [rad]  
(min:  $-\pi/2$ , max:  $+\pi/2$ ).
- **attitude/roll-rad** – Roll of the aircraft (tilt of the wings) [rad]  
(min:  $-\pi$ , max:  $+\pi$ ).
- **velocities/u-fps** – Body frame x-axis velocity [ft/s].
- **velocities/v-fps** – Body frame y-axis velocity [ft/s].
- **velocities/w-fps** – Body frame z-axis velocity [ft/s].
- **velocities/p-rad\_sec** – Roll rate (angular velocity) [rad/s]  
(min:  $-2\pi$ , max:  $+2\pi$ ).
- **velocities/q-rad\_sec** – Pitch rate (angular velocity) [rad/s]  
(min:  $-2\pi$ , max:  $+2\pi$ ).
- **velocities/r-rad\_sec** – Yaw (heading) rate (angular velocity) [rad/s]  
(min:  $-2\pi$ , max:  $+2\pi$ ).

### Calculated variables

- **custom/waypoint/dist\_ft** – Euler distance to active waypoint.
- **custom/waypoint/heading\_error\_rad** – Yaw/heading error to active waypoint.
- **custom/waypoint/elevation\_error\_rad** – Pitch/elevation error to active waypoint.

## 1.2 Action space

It is necessary to give the model full control of the plane’s positioning in order to navigate a 3D environment. However it is not of essence for the model to control the velocity of the plane. Instead the thrust can be set at a default value. This has been done in order to ease the training of the model from the start.

- **fcs/aileron-cmd-norm** – Normalized control of the ailerons.
- **fcs/elevator-cmd-norm** – Normalized control of the elevator.
- **fcs/rudder-cmd-norm** – Normalized control of the rudder.

## 1.3 Environment

We use JSBSim environment connected to OpenAI Gymnasium with visualizations in Flightgear.

## 1.4 Evaluation

Each new task contains a set of waypoints which the model has to fly through with certain accuracy (hit radius is originally set at 50 feet). The evaluation is therefore divided into two variables which both have different priority:

- **Number of waypoints successfully cleared**
- **Distance to active waypoint (if any)**

Let’s say we create a task with one waypoint and we have three models that were trained on it. The first one clears the waypoint, the second one does not and its distance to the waypoint is 400 ft, the third one does not and its distance to the waypoint is 450 ft. The first one is the best since it cleared the most waypoints, the second and third one cleared the same amount of waypoints (0) and thus we compare the distances to the last active waypoint ( $400 < 450$  – the second one is better than the third).

## 2 PPO

Using **PPO** - Proximal Policy Optimization, we were able to achieve promising results for stage 1 quickly in under 500k steps. However, during the second stage of learning, we observed that agent learned to maintain level flight but failed to pursue heading objectives. To address this, we implemented a learning rate with a linear schedule (starting at  $10^{-4}$  and ending at  $10^{-6}$ ). While lower learning rates slowed initial progress, they prevented the policy from "collapsing" into a local optimum where the agent remains static to avoid penalties.

Since PPO does not use an external noise buffer like DDPG, it relies on entropy to drive exploration. The entropy coefficient (`ent_coef`) was tuned to force the agent to "wiggle" the control surfaces. Initially, a low entropy coefficient led to frozen ailerons. By increasing the coefficient to 0.02, the agent began exploring banking maneuvers. However, excessive entropy led to erratic flight paths, indicating a delicate balance is required between exploration and the exploitation of the stability rewards.

## 3 SAC

To overcome the local optima encountered with PPO, we transitioned to **SAC** - Soft Actor-Critic. Unlike PPO, SAC is an off-policy algorithm that features built-in entropy maximization. This characteristic makes the agent continuously explore alongside reward maximization, actively penalizing overly safe or static behaviors. This mechanism proved critical; SAC's aggressive exploration successfully broke the agent out of the aimless flying loop observed in PPO, forcing it to actively discover the banking and turning maneuvers required to intersect the 3D waypoint objectives.

While SAC demonstrated superior sample efficiency and rapidly maximized the reward function, the continuous exploration initially resulted in severe oscillation. The agent exhibited "bang-bang" control behavior, rapidly overcorrecting the ailerons and elevators. To mitigate this, we introduced an action repeat mechanism, running the simulation physics multiple times per single neural network inference. Combined with a distance reward function, this allowed the aircraft dynamics to settle between control inputs. Ultimately, SAC converged on a highly precise and aggressive flight policy, successfully solving the navigation task while minimizing course completion time.

## 4 Memory-Enhanced Agent Approaches

The current PPO implementation uses a standard multilayer perceptron policy (`MlpPolicy`) that is fundamentally **memoryless** – each decision is based solely on the current observation, without consideration of the temporal sequence of past states. For complex navigation tasks such as waypoint following, this lack of memory can limit the agent’s ability to learn temporal dependencies. We consider three approaches:

### 4.1 PPO with LSTM (RecurrentPPO)

The recommended approach utilizes the `sb3-contrib` `RecurrentPPO` implementation of the `sb3-contrib` library, replacing the standard MLP policy with an LSTM-based recurrent policy (`MlpLstmPolicy`). This adds a Long Short-Term Memory layer that maintains an internal hidden state across time steps, allowing the agent to remember past observations within an episode. Implementation requires only changing the policy type and adding LSTM configuration parameters (`lstm_hidden_size`, `n_lstm_layers`). This approach remains model-free, but gains temporal memory for tracking waypoint sequences.

### 4.2 SAC with LSTM

An alternative uses Soft Actor-Critic with an LSTM policy. SAC is an off-policy algorithm that often performs better for continuous control and has better exploration properties. However, switching to SAC requires more substantial pipeline changes, including different hyperparameters. This is better suited as a fallback if PPO+LSTM does not yield satisfactory results.

### 4.3 Model-Based Methods

For a fully model-based solution, algorithms such as DREAMER or MuZero learn an explicit internal model of the aircraft’s dynamics, enabling planning over longer horizons. However, they require significantly more complex implementation and deviate from the `stable-baselines3` framework. Given the goal of minimally invasive changes, **PPO with LSTM** is the recommended approach.

## 5 Waypoint system and reward function

### 5.1 Difficulty based curriculum learning

The waypoints are randomly generated at the start of each episode based on the difficulty value (from 0 to 1). This difficulty value changes the distance and the angle of the waypoint being generated from the aircraft. If the difficulty value is zero, the waypoint is generated at a fair distance right in front of the aircraft (the aircraft itself just needs to keep the current altitude and current heading to get to it). As the value progresses towards one, the distance from the waypoint gets shorter and the waypoint is generated at an angle considering the heading of the aircraft. At the end of each episode the mean reward is calculated and when it is large enough, the difficulty increases slightly so that the aircraft gets a more challenging task on the next episode.

### 5.2 Exploration and exploitation

During the experimental phase it was necessary to find a good reward function. The aircraft does not just need to solve the problem of getting to the next waypoint, from the beginning it mainly needs to solve the "how to stay alive" problem.

Not all positions of the aircraft are allowed in the simulation environment. If the agent tries to head straight up, the simulation will output infinity values and crash. If it heads straight down, it will hit the ground. If it rolls too much, it will not be able to stop. So first of all it is mandatory to set up boundaries for the agent. If the roll or pitch of the aircraft are above 28 degrees, the episode will end with a large penalty for the agent. If the agent crosses the altitude boundary of 2000 ft or 25000 ft, the episode will also end with a large penalty.

With all the extreme edge cases resolved we started constructing the reward function based on the **distance in feet to the next waypoint**, **heading error in radians to the next waypoint** and **elevation error in radians to the next waypoint**. We also added a large reward for when the agent successfully clears the next waypoint and at the end for when it clears all of the waypoints. As soon as we started training, the agent developed a rather strange behavior which can be seen in figure 1. Instead of heading straight for the waypoint, it rather goes away from the waypoint gaining loss and then it returns to get rewarded for getting closer to the waypoint. Another loophole that the agent found can be seen in figure 2. This time it circles around the waypoint to yet again maximize the reward without even touching the waypoint.

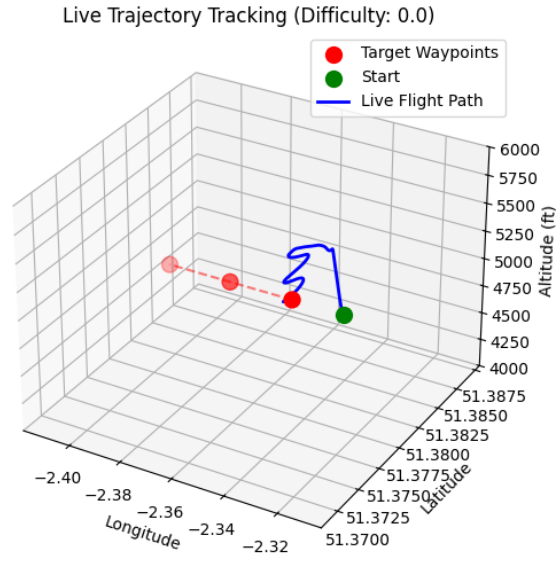


Figure 1: Strange zig zag behavior of the agent exploiting the reward function.

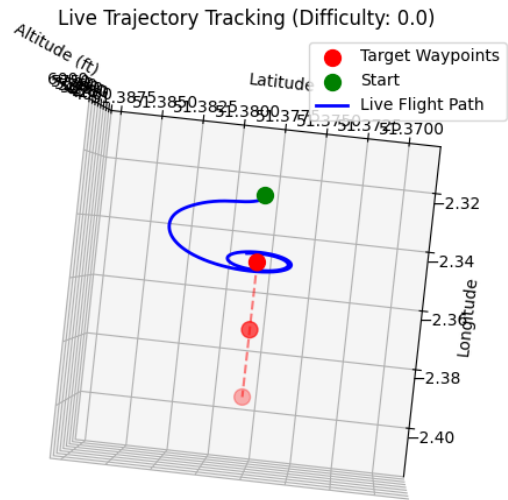


Figure 2: Yet another example of a strange behavior from the agent exploiting the reward function.

## 6 Visualization of the learning process

In order to better visualize the learning process of agent's behavior we created a checkpoint system. After every 50000 time steps we save the currently learned model which we can then visualize.

The visualization is done in two separate environments, the **Python 3D plot** (where the axis are longitude, latitude and altitude) and in **FlightGear** itself. The Python 3D plot shows the waypoints generated for the current episode and the trajectory of the aircraft (see figure 3).

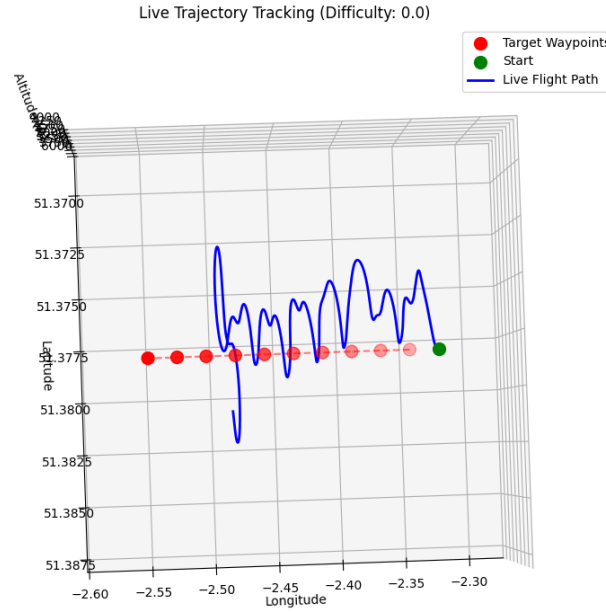


Figure 3: Python 3D plot visualization of the agent exploiting the large hit radius of the waypoint.

When visualization is required we internally send the data concerning the simulation to the Flightgear application via the native FDM protocol. The Flightgear itself is fully dependent on these data and only acts as a visualizer of actions. For waypoint visualization we used the built-in generic protocol. We designed the waypoint model in Blender and converted it to the AC3D format supported by FlightGear. Then we created an XML file which references this model, disables the collision and colors the waypoint. The visualization of the waypoint along with the Cessna 172 P aircraft can be seen in figure 4.





Figure 4: Visualization of agent's behavior with a ten-metre wide waypoint using FlightGear.

## 7 Aircraft Generator and Domain Randomization

A key challenge in training a generalizable flight control agent is the risk of overfitting to a single aircraft model. An agent trained exclusively on the Cessna 172P (C172P) will learn to exploit the precise aerodynamic characteristics of that model rather than develop transferable control principles. To address this, we designed and implemented a procedural aircraft generator that applies domain randomization to the physical parameters of the base aircraft, producing a diverse family of aircraft variants for training.

The generator modifies the JSBSim XML configuration of the C172P model at runtime by scaling a set of physically meaningful parameters. Parameters subject to randomization include the wing area, wingspan, chord length, horizontal and vertical tail surface areas, empty weight, engine power output, zero-lift drag coefficient ( $CD_0$ ), lift curve slope, and pitch moment coefficient. Each parameter is independently sampled from a predefined continuous range centered around the baseline value (e.g., wing area factor  $\in [0.7, 1.4]$ ), producing aircrafts that are aerodynamically distinct but remain within a physically plausible envelope. Moment-of-inertia tensors are derived analytically from the sampled mass and geometry factors, ensuring internal consistency of the generated model.

To avoid redundant file I/O, each generated configuration is identified by an MD5 hash of its parameter vector. If a model with identical parameters already exists on disk, it is reused rather than regenerated. Generated aircraft are stored in an isolated temporary directory with symlinks to the shared engine and systems dat from the JSBSim package, making each instance self-contained and compatible with parallel training environments.

The `RandomAircraftWrapper` class, implemented as a standard Gymnasium wrapper, injects a newly sampled aircraft into the JSBSim environment at each episode reset by patching the environment’s internal simulation factory. This design is non-invasive – it does not require modifications to the underlying `gymnasium-jsbsim` library – and is composable with the curriculum task wrappers. The intended usage is to place the `RandomAircraftWrapper` as the innermost wrapper directly around the base JSBSim environment before any task-specific logic is applied.

The aircraft generator has been implemented and tested in isolation. Full integration with the curriculum training pipeline is planned as part of the final project phase. Once integrated, the agent will encounter a different variant of the aircraft at the start of every episode, which we hope to encourage the development of robust model-agnostic control strategies. We intend to quantitatively evaluate the effect of domain randomization by comparing the transfer performance of a randomization-trained agent against a single-model baseline when evaluated in unseen aircraft configurations.